German Accident Open Data Explorer

PRESENTATION & ORAL DEFENSE GUIDE • DBW EXAM 2026

Student / Presenter:	Samay Patel (Matriculation No. 916936)
Subject / Course:	Datenbanken und Web-Techniken (DBW)
Institution / Examiners:	Technische Universität Chemnitz (Prof. Dr. Michael Martin, Florian Hahn)
Presentation Files:	German_Accident_Data_Explorer_v2.pptx (Saved in Project & Downloads)

1. Overview of Generated PowerPoint Slides (16:9 Widescreen)

Slide #	Slide Title	Core Focus & Academic Contents
Slide 1	Title Slide	Project Title, Student Info (Samay Patel - 916936), Course, Examiners, Tech Stack badges.
Slide 2	Executive Summary	Project Motivation, Problem Statement, 3 Core Pillars (ETL, Schema, API + SPA UI).
Slide 3	Data Sources & Licensing	Ingested datasets (Unfallatlas, Destatis GV-ISys), 8-digit AGS keys, dl-de/by-2-0 licensing.
Slide 4	3-Tier System Architecture	4 Visual Data Flow Blocks: Raw CSVs → import_data.py ETL → PostgreSQL → FastAPI & SPA UI.
Slide 5	Database Schema Design	Relational model in PostgreSQL (regions & accidents tables), 8-digit AGS padding & spatial floats.
Slide 6	Automated ETL Pipeline	Step-by-Step ETL Execution (import_data.py: Extract, DDL, Structural Baseline, Idempotent Load).
Slide 7	REST API Backend Engine	FastAPI ASGI architecture, Pydantic schemas, error handling, Swagger UI (/docs), core routes map.
Slide 8	Glassmorphism Dashboard	Frontend SPA aesthetic, Tailwind CSS #090d16 dark theme, Vanilla JS async fetch(), Modal JSON viewer.
Slide 9	Mandatory Scenarios	Live verification of 5 + 2 mandatory exam questions (Earliest year, Saxony, Berlin, NRW, MV, rates/100k).
Slide 10	Grader Bonus Challenges	Bonus Scenario A (Dresden Bicycle 2024), Bonus Scenario B (Top 5 Fatal Districts), Zero-case municipal logic.
Slide 11	Oral Defense & Demo Plan	Minute-by-minute breakdown (0:00 - 10:00) for managing presentation and live software demonstration.
Slide 12	Conclusion & Compliance	Academic grading rubric verification, < 10 MiB OPAL compliance, zero hardcoding guarantee, future roadmap.

2. Slide-by-Slide Spoken Defense Script (10-Minute Presentation)

Read or practice these spoken lines live during your 10-minute presentation to Prof. Dr. Martin and Mr. Hahn:

Slide 1: Title Slide & Introduction (0:00 - 0:45)

Spoken Script: *"Good morning Prof. Dr. Martin and Mr. Hahn. My name is Samay Patel, matriculation number 916936, studying Master's in Web Engineering at TU Chemnitz. Today I am presenting my project for Datenbanken und Web-Techniken titled 'German Accident Open Data Explorer'. In this project, I have developed an automated ETL data integration pipeline and a dynamic 3-tier web service engine that harmonizes heterogeneous official German open data from the Unfallatlas and Destatis regional registries."*

Slide 2: Problem Statement & Objectives (0:45 - 1:30)

Spoken Script: *"Official German open data is highly valuable but fragmented across multiple CSV files, different regional keys, and varying granularities. To turn this into a usable analytic service, my project focuses on three core pillars: 1. An automated Python ETL pipeline that extracts, normalizes, and bulk-loads accident records. 2. A custom relational database schema in PostgreSQL that cleanly separates demographic dimensions from metric event facts using foreign keys. 3. A high-performance REST API built with FastAPI, auto-documented via OpenAPI/Swagger, and presented through an interactive glassmorphism Web UI."*

Slide 3: Data Sources & Legal Licensing (1:30 - 2:15)

Spoken Script: *"We integrate two primary official open data sources: First, Unfallatlas, providing point-level accident records for 2023 and 2024 with spatial coordinates, severity categories, light conditions, and participant flags for cars, bicycles, and pedestrians. Second, Destatis Gemeindeverzeichnis (GV-ISys), providing official 8-digit Amtlicher Gemeindeschlüssel (AGS) keys and population baselines for states like Saxony, Berlin, NRW, and cities like Chemnitz and Dresden. All processing complies strictly with the Data license Germany – attribution – version 2.0 (dl-de/by-2-0). Every API output payload automatically embeds this legal attribution."*

Slide 4: 3-Tier System Architecture (2:15 - 3:00)

Spoken Script: *"Our system architecture follows a strict 3-tier web model: Raw open data ZIP archives are processed by our import_data.py ETL script. The structured data is stored in a local PostgreSQL relational database (dbw_project). FastAPI reads from the database and exposes REST endpoints. The single-page frontend never connects directly to PostgreSQL—it queries the FastAPI endpoints using asynchronous JavaScript fetch requests. Crucially, zero answers are hardcoded; every metric is computed dynamically in SQL at request time."*

Slide 5: Relational Schema & Key Harmonization (3:00 - 3:45)

Spoken Script: *"Our database schema in PostgreSQL uses two core tables: The regions dimension table stores the 8-digit ags primary key, region name, level, and population. The accidents fact table stores individual accident attributes, WGS84 float coordinates, and references regions(ags) via a foreign key. For harmonization, raw accident columns (ULAND, UREGBEZ, UKREIS, UGEMEINDE) are padded using .zfill() and concatenated into unified 8-digit AGS strings (e.g. '14612000' for Dresden). Comma-separated coordinate strings are converted to standard numeric floats, and missing municipality records are auto-created to prevent foreign key errors."*

Slide 6: Automated ETL Ingestion Execution (3:45 - 4:30)

Spoken Script: *"The ETL execution inside import_data.py follows four automated steps: 1. Extracts raw .zip archives into staging folders. 2. Initializes database table structures using DDL queries. 3. Ingests federal structural metadata and population baselines. 4. Iterates raw CSV files, transforms attributes, and loads rows into PostgreSQL. The ingestion pipeline uses ON CONFLICT DO UPDATE and conflict-safe transactions, making it completely idempotent and safe to re-run at any time without duplicate data."*

Slide 7: FastAPI REST API & Interactive OpenAPI Docs (4:30 - 5:15)

Spoken Script: *"Our backend engine uses FastAPI with asynchronous ASGI execution and Pydantic schema validation. Interactive OpenAPI documentation is automatically served at /docs for examiner live testing. Key routes include /metadata/earliest-year for temporal boundaries, /aggregates/accidents for dynamic filtering by AGS and year, /regions for administrative metadata, and specialized /custom/ endpoints for sub-municipal analytical queries."*

Slide 8: Glassmorphism Web Dashboard (5:15 - 6:00)

Spoken Script: *"The frontend is a single-page application designed with modern dark-mode glassmorphism aesthetics using Tailwind CSS and Google Inter typography. Using asynchronous fetch(), clicking any scenario card immediately queries the backend without full-page reloads. The dashboard includes an interactive JSON modal viewer that overlay-renders raw structured API responses, offering complete transparency for verification."*

Slide 9: Verification of Mandatory Examiner Scenarios (6:00 - 7:00)

Spoken Script: *"Our service dynamically verifies all mandatory exam scenarios from real PostgreSQL database data: Q1 (Earliest Year): Computed via /metadata/earliest-year (2023). Q2 (Saxony 2023 Injuries): Queried via /aggregates/accidents?ags=14&year;=2023. Q3 (Berlin 2023 Pedestrians): Filtered via /aggregates/accidents?ags=11&year;=2023&pedestrian;_only=true. Q4 & Q5 (NRW & MV Start Boundaries): Evaluated dynamically via minimum year queries on AGS prefixes 05% and 13%. Q6 & Q7 (Multi-Dataset Integration): Calculates accident rates per 100,000 inhabitants by combining accidents counts with regions population figures."*

Slide 10: Grader Bonus Challenges & Analytical Depth (7:00 - 8:00)

Spoken Script: *"Beyond mandatory requirements, we implemented two grader bonus challenges: Bonus A (Dresden Bicycle Accidents 2024): Filters sub-municipal AGS 14612% with ist_rad = 1 via /custom/dresden-bicycle-2024. Bonus B (Top 5 Fatal Districts 2024): Executes advanced SQL grouping and string slicing (GROUP BY SUBSTRING(ags,1,5)) via /custom/top-fatal-districts-2024. We also support zero-case municipal checks by cross-referencing complete regions records against accidents events."*

Slide 11: Live Demonstration Plan (8:00 - 9:15)

Spoken Script: *"Now, let us move to the live application demonstration. [Switch window to browser at http://127.0.0.1:8000/] Here on our dashboard, we can click 'Execute Query' for any scenario. Notice how the modal instantly pops up showing the raw JSON payload with provenance metadata. Next, visiting http://127.0.0.1:8000/docs, we can execute /custom/top-fatal-districts-2024 live inside Swagger to verify dynamic SQL execution."*

Slide 12: Academic Evaluation & Conclusion (9:15 - 10:00)

Spoken Script: *"In summary, the project satisfies 100% of the course requirements: Official German open data is integrated into a custom relational schema. A documented REST API serves live calculated answers. The entire submission strictly respects the 10 MiB limit on OPAL without unnecessary node_modules bloat. Thank you Prof. Dr. Martin and Mr. Hahn for your attention. I welcome your questions!"*

3. Step-by-Step Live Software Demonstration Roadmap

Perform these 3 preparation steps before booting your presentation for Prof. Dr. Martin:

- **Step 1: Ensure PostgreSQL Service is Active:** Verify PostgreSQL port 5432 is running locally with database dbw_project authenticated as user postgres.
- **Step 2: Start Uvicorn FastAPI Server:** cd d:\dbw_Project

```
python -m uvicorn app:app --reload --port 8000
```

- **Step 3: Open Browser Demonstration Tabs:** Tab 1: http://127.0.0.1:8000/ (Main Glassmorphism Single-Page Dashboard)

```
Tab 2: http://127.0.0.1:8000/docs (Interactive Swagger OpenAPI Documentation)
```

4. Defense Q&A; Preparation (Answering Examiner Questions)

Q1: Why did you use an 8-digit AGS key instead of standard text names?

Answer: *"Official German administrative boundaries are hierarchical: State (2 digits), District (1 digit), County (2 digits), and Municipality (3 digits). Text names like 'Chemnitz' or 'Dresden' can have formatting variations across datasets. An 8-digit padded string guarantees exact relational joins between accident event records and demographic population tables."*

Q2: How do you handle missing municipalities in the regions table when loading accidents?

Answer: *"In import_data.py, before inserting an accident record, the script checks if the target ags exists in regions. If missing, it dynamically executes an INSERT INTO regions (ags, name, level) VALUES (..., 'Region_...', 'municipality') ON CONFLICT DO NOTHING; to guarantee referential integrity without throwing foreign key exceptions."*

Q3: How did you ensure your project stays below the 10 MiB OPAL upload limit?

Answer: *"By using native Python libraries (FastAPI, Psycopg2, Pandas), delivering frontend CSS/JS via lightweight CDN links (Tailwind CDN, Google Fonts), and storing data in PostgreSQL rather than bundling heavy static JSON databases into the submission ZIP."*

Q4: Are the API answers cached or hardcoded?

Answer: *"No, every endpoint executes live SQL queries (psycopg2 cursor calls) directly against PostgreSQL at request time. Re-running the ETL or adding new annual accident records automatically updates all API responses without code changes."*